

SIMPLY FPU

by **Raymond Filiatreault**
Copyright 2003

Chap. 1 Description of FPU Internals

The FPU simply has eight identical 80-bit registers and three 16-bit registers ([Control](#) Word, [Status](#) Word and [Tag](#) Word) accessible to the programmer. It also has an internal [flag](#) register which is not accessible to the programmer.

80-bit Registers

The 80-bit registers are generally designated in most literature as a **stack** of eight registers. To better understand how these 80-bit registers function, instead of imagining them as a stack, it will be easier to **imagine** them as a *revolver barrel* with 8 compartments numbered clock-wise from 0 to 7. When the FPU is initialized, all the compartments are empty and Barrel Compartment #0 (BC0) would be at the 12 o'clock position (at the **TOP**), as depicted in Fig.1.1 below.

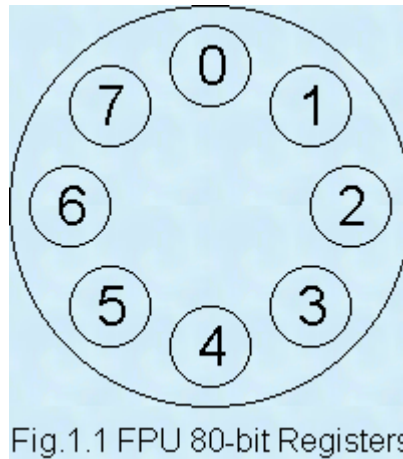


Fig.1.1 FPU 80-bit Registers

Note: These **Barrel Compartment (BC)** numbers are strictly for internal use by the FPU. They are used to identify each compartment as a physical unit similarly to the EAX, EBX, etc. registers in the CPU. (There is NO revolving barrel nor any stack as such in the FPU.)

When the FPU would be instructed to **LOAD** a value, it would turn the barrel clockwise by one notch and load the specified value in the top compartment. The first value loaded immediately after the FPU is initialized would thus go into BC7 according to the FPU's internal numbering system.

If the FPU would be instructed to load another value while the first one is still in BC7, it would again turn the barrel clockwise by one notch and load the specified value again in the top compartment, which would now be BC6.

Values can be loaded only into the TOP compartment of the FPU

This could continue until all the compartments contain a value. If, however, an attempt is made to load a value when all the compartments have a value in them, the barrel would still turn by one notch but the attempted loading would fail (just like trying to insert a bullet into a compartment which already contains one). And, in addition, whatever valid value would have been in that compartment now at the TOP is also destroyed, leaving unusable trash in that register at the TOP.

Rule #1: An FPU 80-bit register compartment MUST be free (empty) in order to load a value into it.

Quite fortunately, these registers can be emptied with various FPU instructions. The most common way is generally referred to as "popping a register". The "pop" mnemonic used for the CPU is not available for the FPU. Instead, it can be included as a part of numerous FPU instructions; such instruction would be carried out normally and then immediately followed by popping the register at the TOP.

When the FPU is instructed to **POP** a value, it would first remove it from whichever compartment would currently be at the TOP and then turn the barrel **counter-clockwise** by one notch. For example, if BC6 would be at the TOP and popped, BC7 would then become the register compartment at the TOP.

Values can be popped only from the TOP compartment of the FPU

Those BC numbers are never used directly by the programmer. The FPU takes care of remembering where all the 80-bit values are located in its internals and which of its compartments is at the TOP. However, the programmer must remain aware of this internal numbering system.

For the programmer, while still using the revolving barrel image, the 80-bit registers are **ALWAYS** numbered clockwise from 0 to 7 **starting from the TOP**. The numbers shown in Fig.1 above would therefore never change for referring to register numbers in FPU instructions. The register at the TOP would always have the number 0.

The designation **ST** with its number in parenthesis (such as **ST(0)**, **ST(1)**, etc.) is used when reference to a given 80-bit register is required in an FPU instruction. (MASM also interprets ST without any explicit number as if ST(0) had been specified.)

Any value loaded to the FPU must initially be referred to as ST(0) because it can only be loaded to the TOP compartment. If the FPU would be instructed to load another value while the first one is still there, that second value would now be referred to as ST(0) ***because it has now become the one at the TOP***. As a consequence, the first value would now have to be referred to as ST(1). If another value is loaded, the first value would then have to be referred to as ST(2). After popping the last loaded value, that same first value would revert back to being referred to as ST(1).

That is probably the most complex concept to understand by someone starting to learn how to use the FPU. When compared to the CPU where a value in EAX would always be referred to as EAX regardless of operations on the other registers, a value in an FPU register must be referred to according to its position relative to the register at the TOP.

Rule #2: The programmer must constantly keep track of the relative location of the existing register values while other values may be loaded to or popped from the TOP register.

A good programming practice is to insert a comment after each FPU instruction which can affect the location of register values, indicating the new ST number of each value.

When a register is popped from the FPU, its current value can no longer be used in any operation. If that value would need to be used later, it should be stored in memory before popping it and reloaded when required. (Some debuggers may still show the old value in the popped register but that should only be considered as residual "gun powder".)

16-bit Registers

The three 16-bit registers available to the programmer are the **control word**, the **status word** and the **tag word**.

Control Word

The Control Word 16-bit register is used by the programmer to select between the various modes of computation available from the FPU, and to define which exceptions should be handled by the FPU or by an exception handler written by the programmer.

The Control Word is divided into several bit fields as depicted in the following Fig.1.2.

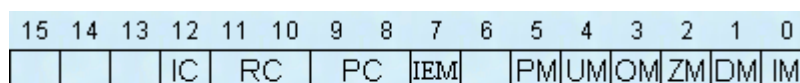


Fig.1.2 Control Word Fields

The **IC** field (bit 12) or **I**nfinity **C**ontrol allows for two types of infinity arithmetic:

0 = Both -infinity and +infinity are treated as unsigned infinity (initialized state)

1 = Respects both -infinity and +infinity

This field has been retained for compatibility with the 287 and earlier co-processors. In the more modern FPU's, this bit is disregarded and both -infinity and +infinity are respected.

The **RC** field (bits 11 and 10) or **R**ounding **C**ontrol determines how the FPU will round results in one of four ways:

00 = Round to nearest, or to even if equidistant (this is the initialized state)

01 = Round down (toward -infinity)

10 = Round up (toward +infinity)

11 = Truncate (toward 0)

The **PC** field (bits 9 and 8) or **P**recision **C**ontrol determines to what precision the FPU rounds results after each arithmetic instruction in one of three ways:

00 = 24 bits (REAL4)

01 = *Not used*

10 = 53 bits (REAL8)

11 = 64 bits (REAL10) (this is the initialized state)

The **IEM** field (bit 7) or **I**nterrupt **E**nable **M**ask determines whether any of the interrupt masks will be enabled (bit = 0) or all those masks will be disabled (bit = 1). This bit field is set to 1 in the initialized state. *(This field is also for compatibility with early co-processors and not used anymore.)*

Bits 5-0 are the interrupt masks. In the initialized state, they are all set to **1** which lets the FPU handle all exceptions. When any one of them is set to **0**, it instructs the FPU to generate an interrupt whenever that particular exception is detected so that the program will take whatever action may be deemed necessary before returning control to the FPU.

The various interrupt masks available are:

PM (bit 5) or **P**recision **M**ask

UM (bit 4) or **U**nderflow **M**ask

OM (bit 3) or **O**verflow **M**ask

ZM (bit 2) or **Z**ero divide **M**ask

DM (bit 1) or **D**enormalized operand **M**ask

IM (bit 0) or **I**nvalid operation **M**ask

(A more detailed description of the various exceptions and how the FPU would normally handle them is given in the following section. This document will not describe how interrupts are generated and transmitted nor how to respond to such interrupts.)

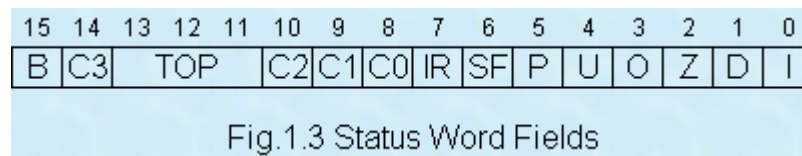
Bits 15-13 and 6 are reserved or unused.

See the [FSTCW](#) and the [FLDCW](#) instructions for details on how to access the Control Word register.

Status word

The Status Word 16-bit register indicates the general condition of the FPU. Its content may change after each instruction is completed. Part of it cannot be changed directly by the programmer. It can, however, be accessed indirectly at any time to inspect its content.

The Status Word is divided into several bit fields as depicted in the following Fig.1.3.



When the FPU is initialized, all the bits are reset to 0.

The **B** field (bit 15) indicates if the FPU is busy (B=1) while executing an instruction, or is idle (B=0).

The **C3** (bit 14) and **C2 - C0** (bits 10-8) fields contain the condition codes following the execution of some instructions such as comparisons. These codes will be explained in detail for each instruction affecting those fields.

The **TOP** field (bits 13-11) is where the FPU keeps track of which of its 80-bit registers is at the TOP. The **BC** numbers described previously for the FPU's internal numbering system of the 80-bit registers would be displayed in that field. When the programmer specifies one of the FPU 80-bit registers $ST(x)$ in an instruction, the FPU adds (modulo 8) the ST number supplied to the value in this **TOP** field to determine in which of its registers the required data is located.

The **IR** field (bit 7) or **Interrupt Request** gets set to 1 by the FPU while an exception is being handled and gets reset to 0 when the exception handling is completed. When the interrupt is masked in the Control Word for the FPU to handle the exception, this bit may never be seen to be set while stepping through the instructions with a debugger. However, if the programmer handles the interrupt, that bit should remain set until the interrupt handling routine is completed.

Bits 6-0 are flags raised by the FPU whenever it detects an exception. Those exception flags are cumulative in the sense that, once set (bit=1), they are not reset (bit=0) by the result of a subsequent instruction which, by itself, would not have raised that flag. Those flags can only be reset by either initializing the FPU ([FINIT](#) instruction) or by explicitly clearing those flags ([FCLEX](#) instruction).

The **SF** field (bit6) or **Stack Fault** exception is set whenever an attempt is made to either load a value into a register which is not free (the C1 bit would also get set to 1) or pop a value from a register which is free (and the C1 bit would get reset to 0). (Such stack fault is also treated as an invalid operation and the **I** field flag bit0 would thus also be set by this exception; see below.)

The **P** field (bit5) or **Precision** exception is set whenever some precision is lost by instructions which do exact arithmetic. For example, dividing 1 by 10 does not yield an exact value in binary arithmetic and would set the **P** exception flag. Another example which sets the **P** exception flag would be the conversion of a REAL10 to a REAL4 when some of the least significant bits would be lost.

If the FPU handles this exception (when the PM bit is set in the Control Word), it rounds the result according to the rounding mode specified in the RC field of the Control Word.

The **U** field (bit4) or **Underflow** exception flag gets set whenever a value is too small (without being equal to 0) to be represented properly. Each of the floating point formats has a different limit on the smallest number which can be represented. The **U** flag gets set if the result of an operation exceeds that limit. For example, dividing a valid very small number by a large number could exceed the limit. A valid REAL10 small number may be much smaller than acceptable for the REAL4 or REAL8 formats; in such cases, conversion from the former to the latter would also set the U flag.

If the FPU handles this exception (when the UM bit is set in the Control Word), it would [denormalize](#) the value until the exponent is in range or ultimately return a 0.

The **O** field (bit3) or **O**verflow exception flag gets set whenever a value is too large in magnitude to be represented properly. Again, each of the floating point formats has a different limit on the largest number which can be represented. The **O** flag gets set if the result of an operation exceeds that limit. For example, multiplying a valid very large number by another large number could exceed the limit. A valid REAL10 large number may be much larger than acceptable for the REAL4 or REAL8 formats; conversion from the former to the latter would also set the **O** flag.

If the FPU handles this exception (when the OM bit is set in the Control Word), it would generate a properly signed [INFINITY](#) according to the IC flag of the Control Word.

The **Z** field (bit2) or **Z**ero divide exception flag gets set whenever the division of a finite non-zero value by 0 is attempted.

If the FPU handles this exception (when the ZM bit is set in the Control Word), it would generate a properly signed [INFINITY](#) according to the XOR of the operand signs and then according to the IC flag of the Control Word.

The **D** field (bit1) or **D**enormalized exception flag gets set whenever an instruction attempts to operate on a [denormalized](#) number or the result of the operation is a denormalized number.

If the FPU handles this exception (when the DM bit is set in the Control Word), it would simply continue with normal processing and then check for other possible exceptions.

The **I** field (bit0) or **I**nvvalid operation exception flag gets set whenever an operation is considered invalid by the FPU. Examples of such operations are:

- Stack overflow or underflow
- Indeterminate arithmetic such as 0 divided by 0, or subtracting infinity from infinity
- Using a Not-A-Number ([NAN](#)) as an operand with some instructions
- Trying to extract the square root of a negative number

If the FPU handles this exception (when the IM bit is set in the Control Word), it would either return the NAN if one was used as an operand (or the larger absolute value if two NANs were used as operands), or otherwise return a special "[INDEFINITE](#)" NAN.

See the [FSTSW](#) instruction for details on how to access the Status Word register.

Tag Word

The Tag Word 16-bit register is managed by the FPU to maintain some information on the content of each of its 80-bit registers.

The Tag Word is divided into 8 fields of 2 bits each as depicted in the following Fig.1.4.

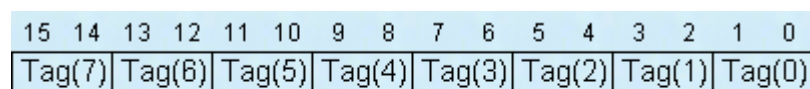


Fig.1.4 Tag Word Fields

The above Tag numbers correspond to the FPU's internal numbering system for the 80-bit registers (the BC numbers). The meaning of each pair of bits is as follows:

- 00 = The register contains a valid non-zero value
- 01 = The register contains a value equal to 0
- 10 = The register contains a special value (NAN, infinity, or denormal)
- 11 = The register is empty

When the FPU is initialized, all the 80-bit registers are empty and the Tag Word would thus have an overall value of

111111111111111b (FFFFh).

If a valid non-zero value is then loaded, the Tag Word would then be change to 001111111111111b (3FFFh). (Remember that the very first value loaded goes into BC7.)

If a second value equal to 0 was then loaded, the Tag Word would become 000111111111111b (1FFFh). (And the second value loaded goes into BC6.)

Although this Tag Word may contain information which could also be useful to the programmer, it cannot be accessed directly nor by itself. The only way to gain access to it is to store the FPU's environment data in memory (see the [FSTENV](#) instruction) and examine it there. However, the information available in the Tag Word could also be obtained otherwise (such as with the [EXAM](#) instruction for individual registers).

Internal flag register

The FPU also has an internal exception flag register which is not accessible to the programmer. All these flags are cleared before each instruction and are set as each exception is encountered. Those are the flags that trigger a response from the FPU or an interrupt for the programmer's exception handler. They are also OR'ed with the exception flags of the Status Word to provide a cumulative record for the programmer.

It is possible that several of the flags could be set with a single instruction. For example, using a denormal number as an operand would set the **D**enormal flag. The result of the operation with it could then set the **U**nderflow flag and the **P**recision flag.

[RETURN TO](#)
[SIMPLY FPU](#)
[CONTENTS](#)